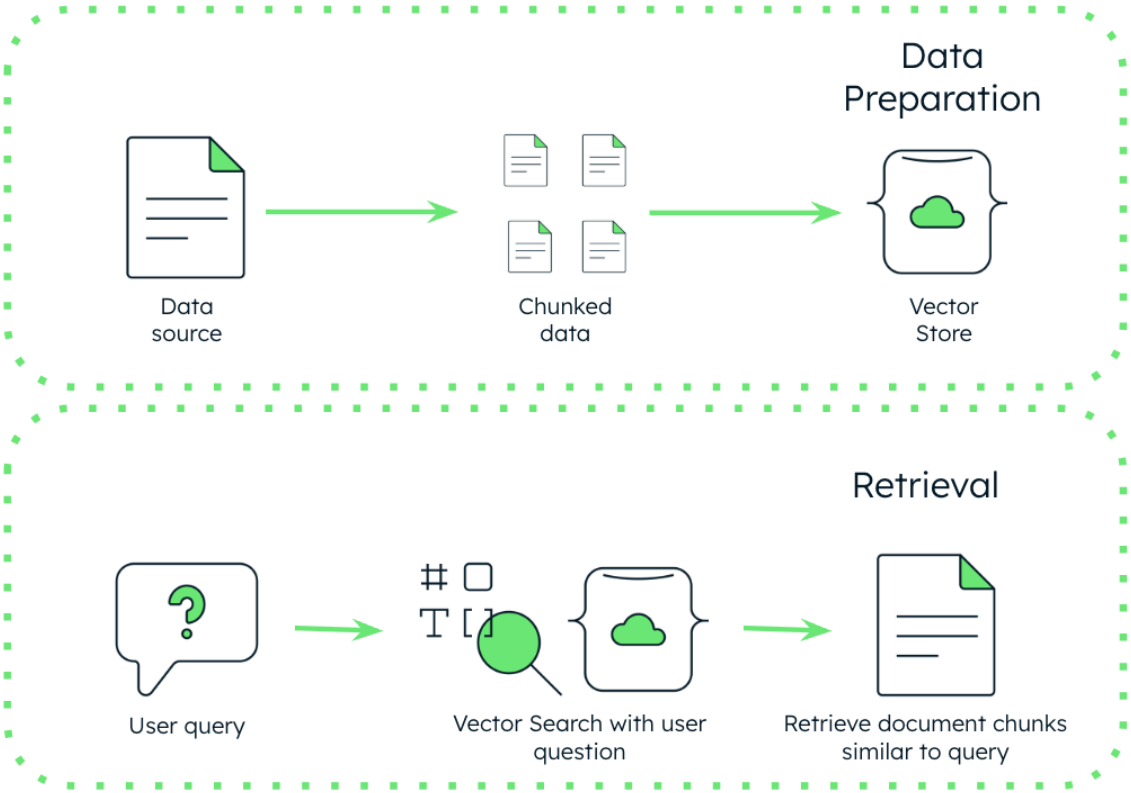
1.RAG Execution Workflow

1. Indexing Phase

a. **Collect data -** PDFs, docs, web pages, DB rows
b. **Chunk the data ***
c. **Create embeddings**
d. **Store in vector DB**

2. Query Phase

a. User asks question
b. Convert question → embedding
c. Vector DB similarity search
d. Retrieve top-k chunks
e. Build augmented prompt
f. LLM generates answer



2. Chunking Strategy

What is Chunking?

— Splitting large documents into **small, meaningful pieces** before embedding.

Why chunk?

- Embedding models have token limits
- Smaller chunks → better semantic match
- Large chunks dilute relevance

What kind of data can be chunked?

You can chunk almost any kind of data. Here are some examples:

- **Text data :** Large text documents, books, and logs can be chunked into smaller paragraphs, sentences, and tokenized units in natural language processing (NLP) and sentiment analysis.
- **Numerical data :** Large datasets, such as tabular or time series data, can be split into smaller subsets or time intervals for easier analysis, visualization, and machine learning model training.
- **Binary data :** Files like software packages and databases can be chunked into blocks for transmission, storage, and deduplication.
- **Image, video, and audio data :** Images, video, and audio can be chunked into smaller segments like image tiles, video frames, and audio samples to enable tasks like compression, streaming, and localized processing.
- **Network or streaming data :** Continuous data streams, such as IoT sensor outputs or real-time traffic logs, can be divided into time-based or size-based chunks for real-time analysis or storage.

Chunking simplifies data handling and enhances performance, scalability, and usability, making it essential for analysis.

Types of chunking

There are several types of data chunking, some of which are:

- **Fixed-size chunking :** In this scenario, data is divided into equal-sized chunks. It's straightforward and ideal for file storage systems, streaming data processing, and **batching** in machine learning.
- **Variable-size chunking :** In this scenario, data is divided into chunks of various sizes. It's ideal for deduplication in storage systems and handling irregular data patterns.
- **Content-based chunking :** In this scenario, data is chunked according to specific patterns within content rather than size. It can generally be used for backup and deduplication systems with similar content.
- **Logical chunking :** With this type of chunking, data is broken down according to logical units rather than size. It processes text by sentences or paragraphs, time series data by time intervals, and database records by keys.
- **Dynamic chunking :** With this type of chunking, data is sized and adjusted based on constraints like memory availability and **workload distribution**. It's ideal for streaming applications, **real-time analytics**, and adaptive systems.
- **File-based chunking :** With this type of chunking, big files are split into smaller pieces for transfer, storage, and processing. It's used for file-sharing systems, cloud storage, and video streaming. An example of file-based chunking is breaking a video into smaller segments for adaptive streaming.
- **Task-based chunking :** With this type of chunking, data is divided into chunks optimized for parallel processing tasks. It's used for parallel training of machine learning models and distributed systems.

What is data chunking used for?

Data chunking solves problems concerning memory limits, data transfer, and processing speed. Here are some of the specific ways it's used:

Optimizing memory usage

Chunking enables systems to handle large datasets without exceeding their memory capacity. Prevention of memory overload allows for efficient operations even on systems with limited resources. For example, in machine learning, data can be processed in small batches during training to avoid overwhelming system resources while ensuring faster and more efficient computations.

Improving data transfer

Chunking improves data transfer by breaking large files into smaller chunks, optimizing bandwidth utilization. This approach reduces downtime during errors because only the corrupted chunk needs to be resent instead of the entire file. Chunking also improves resiliency in the face of bandwidth limitations and ensures smoother, more reliable data transfers over networks.

Parallel processing of data

Chunking enables large datasets to be divided into smaller chunks that can be processed simultaneously across multiple processors or nodes. Each chunk is handled independently, allowing tasks to run in parallel, reducing overall processing time and improving efficiency. After processing, the individual chunks are combined to produce a unified result.

Retrieval-Augmented Generation (RAG) and Large Language Models (LLMs)

Data chunking is essential in RAG frameworks and LLMs because it manages big datasets and optimizes processing within fixed token limits. In RAG, large documents are divided into smaller, semantically coherent chunks that can be efficiently indexed and retrieved. When a query is made, only the most relevant chunks are fetched and passed to the LLM, ensuring precise and contextually relevant responses. Overall, chunking enhances retrieval accuracy, reduces latency, and allows seamless handling of complex queries.

Chunking strategies

Your chosen strategy depends on the data type, use case, and intended outcome. Here’s a look at some common chunking strategies:

- **Batch Processing**: Divides large datasets into smaller batches that can be processed sequentially. Every batch provides another incremental piece, updating systems iteratively.
- **Windowing**: A chunking technique where a continuous data stream is divided into smaller chunks called windows. This strategy enables real-time analysis and pattern detection because it processes each window independently.
- **Distributed Chunking**: Splits data for processing across multiple nodes. By allowing chunks to be processed independently, you improve fault tolerance, scalability, and efficiency.
- **Hybrid Strategies**: Combines several chunking strategies for scenarios with complex requirements. For example, you can utilize fixed-size and logical chunking to divide video files into fixed-size chunks while preserving scene boundaries for seamless playback and analysis.
- **On-the-Fly Chunking**: Instead of chunks being predefined, this strategy spits them on the fly during processing. This works well for real-time applications like live streaming or sensor data processing.

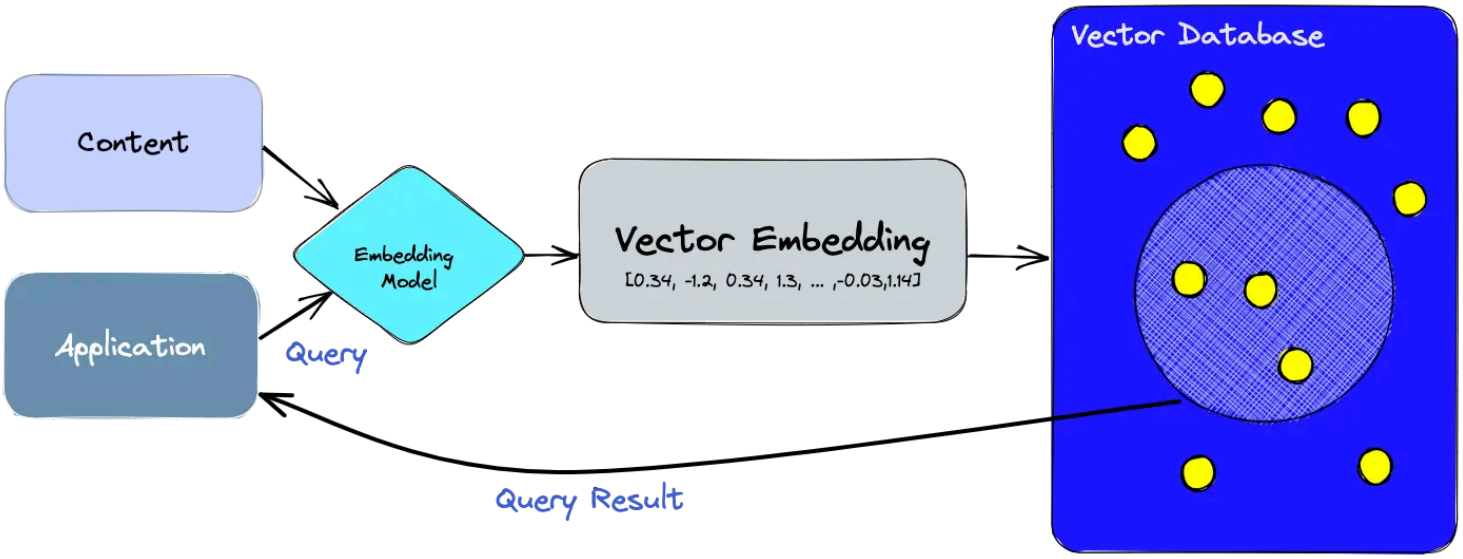
3. Local & Free Vector Databases

What is a Vector DB?

- Stores vectors (embeddings)
- Supports **similarity search** (cosine / dot product)
- Optimized for high-dimensional data

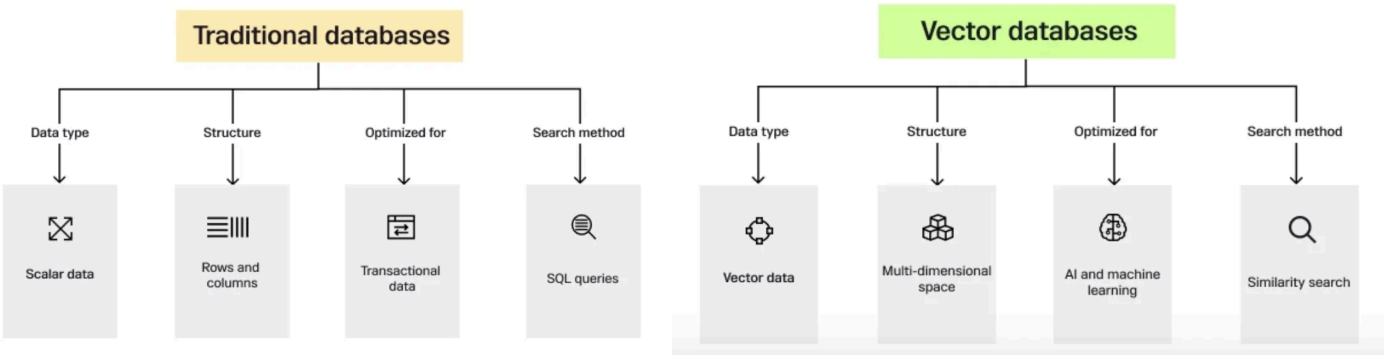
What are Embeddings?

- Embeddings are dense numerical representations of data such as words, sentences, images or audio mapped into a continuous high dimensional space where similar items are positioned closer together.
- Machine learning models that capture semantic meaning, context and relationships within the data generates them.
- Instead of comparing raw text or media directly embeddings allow systems to measure similarity through mathematical distance metrics like cosine similarity or Euclidean distance for faster search and extraction.
- This makes them important for tasks such as semantic search, recommendation systems, clustering, classification and cross lingual matching.



Popular FREE / LOCAL Options

Vector DB	Best for
FAISS	Fast, local, research
ChromaDB	Beginner-friendly, Python
Milvus	Production-scale
Weaviate (OSS)	Graph + vector
Qdrant	Rust-based, very fast



4. Advanced RAG

Advanced RAG techniques for developers

Advanced Retrieval-Augmented Generation (Advanced RAG) extends the basic RAG pipeline to improve **accuracy, robustness, scalability, and reasoning quality**.

It is designed for real-world systems where data is large, noisy, ambiguous, or mission-critical.

While basic RAG focuses on *retrieving and generating*, advanced RAG focuses on **retrieving smarter and generating more reliably**.

Why Advanced RAG is Needed

Basic RAG systems often face the following limitations:

- Irrelevant or partially relevant retrieved chunks
- Token limit constraints
- Poor handling of ambiguous queries
- Lack of reasoning across multiple documents
- No self-verification of generated answers

Advanced RAG techniques address these issues by improving retrieval strategies, context handling, and reasoning workflows.

---

1. Hybrid Search RAG

Description

Hybrid Search RAG combines:

- **Vector-based semantic search**
- **Keyword-based lexical search (e.g., BM25)**

The final retrieval result is produced by merging or scoring results from both approaches.

Benefit

- Improved recall and precision
- Better performance on technical and structured queries

---

2. Re-Ranking (Two-Stage Retrieval)

Description

Re-ranking introduces an additional evaluation step after initial retrieval.

Workflow

1. Retrieve top *N* documents (e.g., 20–50) using fast vector search
2. Re-score each document using:
    - Cross-encoder models, or
    - LLM-based relevance scoring
3. Select top *K* documents (e.g., 3–5) for generation

Benefit

- Higher-quality context
- Significant reduction in hallucinations

---

3. Multi-Query RAG

Description

Multi-Query RAG generates multiple semantically equivalent queries from a single user question.

Each generated query is used independently for retrieval, and the results are combined.

Benefit

- Higher recall
- Better coverage of relevant information
- Improved robustness to query formulation

---

4. Context Compression RAG

Description

Context Compression reduces the size of retrieved documents before passing them to the LLM.

Techniques

- Extractive summarization
- Abstractive summarization
- Sentence-level relevance filtering

Benefit

- Lower token usage
- Reduced noise
- Improved focus on relevant information

---

5. Self-Querying RAG

Description

In Self-Querying RAG, the LLM parses the user query to identify:

- Intent
- Constraints
- Metadata filters

These are then applied during retrieval.

Example Capabilities

- Date filtering
- Document type filtering
- Author or category filtering

Benefit

- More precise retrieval
- Better handling of structured or conditional queries

---

6. Agentic RAG

Description

Agentic RAG allows the LLM to act as an **autonomous agent** that decides:

- When retrieval is needed
- Which tool to use
- Whether multiple retrieval steps are required

Characteristics

- Tool usage (Vector DBs, APIs, Web search, Databases)
- Iterative reasoning loops
- Decision-based workflows

Benefit

- Supports complex, multi-step questions
- Enables adaptive information gathering

---

7. Self-Reflective / Self-Correcting RAG

Description

The model evaluates its own output for confidence, completeness, or factual grounding.

If the answer is insufficient:

- Additional retrieval is triggered
- The response is refined

Benefit

- Reduced hallucination rate
- Improved answer reliability
- Better alignment with source material

8. Graph RAG

Description

Graph RAG represents knowledge as:

- **Entities**
- **Relationships**

Instead of retrieving flat text chunks, the system retrieves connected knowledge paths.

Benefit

- Improved reasoning across concepts
- Better answers for causal and relational questions

---

9. RAG Evaluation and Observability

Description

Advanced RAG systems include evaluation mechanisms to measure performance.

Common Metrics

- Retrieval precision and recall
- Groundedness of responses
- Faithfulness to retrieved context
- Hallucination frequency

Importance

Evaluation enables systematic improvement and production readiness.

Summary

Advanced RAG focuses on improving retrieval quality and reasoning depth, not merely increasing model size.